

Московский авиационный институт (национальный исследовательский университет)

Институт «Компьютерные науки и прикладная математика»

Кафедра «Вычислительная математика и программирование»

Лабораторная работа №1

по курсу «Дискретный анализ»

на тему «Сортировки за линейное время»

Студент: Строков Георгий Владимирович

Группа: М8О-208БВ-24

Преподаватель: Макаров Н.К.

Проверил: Сахарин Н.А.

Дата: 20 марта 2026 г.

Оценка: _____

Подпись: _____

Москва, 2026

1 Постановка задачи

Требуется разработать программу, осуществляющую ввод пар «ключ-значение», их упорядочивание по возрастанию ключа указанным алгоритмом сортировки за линейное время и вывод отсортированной последовательности. Алгоритм сортировки: поразрядная сортировка.

2 Формат ввода

На каждой непустой строке входного файла располагается пара «ключ-значение», в которой ключ указан согласно заданию, затем следует знак табуляции и указано соответствующее значение. Каждая строка содержит:

- Тип ключа: даты в формате DD.MM.YYYY.
- Тип значения: числа от 0 до $2^{64}-1$.

Пары разделены пробельными символами.

3 Формат вывода

Выходные данные состоят из тех же строк, что и входные, за исключением пустых и порядка следования. Порядок следования определяется алгоритмом сортировки, согласно условию задания.

4 Метод решения (Алгоритм)

Используется алгоритм **Radix sort** (поразрядная сортировка):

1. На вход алгоритма подается два итератора - начало и конец контейнера.
2. Алгоритм выполняется целое число раз. Число равно целочисленному делению вверх от размера типа сортируемых данных на шаг - число, сколько бит алгоритм сортирует за одну итерацию.
3. Объект класса `Digitizer` для каждой итерации возвращает число, которое равно размеру буфера, и позицию для каждого элемента исходного массива в этом буфере.
4. Алгоритм проходит по исходному массиву и для каждого элемента увеличивает значение в буфере на единицу, ключом которого является текущий разряд числа.
5. Алгоритм проходит по буферу и для каждого индекса строит префиксную сумму $buffer[i] = buffer[i - 1]$.
6. Алгоритм проходит с конца по исходному массиву, и каждый элемент записывает на соответствующую позицию в выходной массив, согласно его позиции в `buffer`.

5 Исходный код

```
1 #include <iostream>
2 #include <c std def>
3 #include <c std int>
4 #include <iterator>
5
6
7 struct Item {
8     uint32_t date;
9     uint32_t pos;
10    uint32_t len;
11 };
12
13
14 template <typename T>
15 class MyVector {
16 private:
17     T* data_;
18     std :: size _t size _;
19     std :: size _t capacity _;
20
21 public:
22     MyVector()
23     : data_{new T[8]() }
24     , size_{0}
25     , capacity_{8}
26     {}
27
28     explicit MyVector( std :: size _t n)
29     : data_{new T[n]() }
30     , size_{n}
31     , capacity_{n}
32     {}
33
34     template <typename Iter>
35     MyVector(Iter start, Iter end)
36     : data_{nullptr}
37     , size_{0}
38     , capacity_{0}
39     {
40         auto dist = std :: distance(start, end);
41         data_ = new T[dist]();
```

```

42         size_ = dist;
43         capacity_ = dist;
44         std::size_t idx = 0;
45         for (auto it = start; it != end; ++it) {
46             data_[idx++] = *it;
47         }
48     }
49
50     MyVector(const MyVector& other)
51     : data_{new T[other.capacity_]}
52     , size_{other.size_}
53     , capacity_{other.capacity_}
54     {
55         for (std::size_t i = 0; i != other.size_; ++i) {
56             data_[i] = other.data_[i];
57         }
58     }
59
60     MyVector(MyVector&& other) noexcept
61     : data_{other.data_}
62     , size_{other.size_}
63     , capacity_{other.capacity_}
64     {
65         other.data_ = nullptr;
66         other.size_ = 0;
67         other.capacity_ = 0;
68     }
69
70     MyVector() noexcept {
71         delete[] data_;
72         size_ = 0;
73         capacity_ = 0;
74     }
75
76     MyVector& operator=(const MyVector& other) {
77         if (this != &other) {
78             delete[] data_;
79
80             data_ = new T[other.capacity_];
81             size_ = other.size_;
82             capacity_ = other.capacity_;
83             for (std::size_t i = 0; i != other.size_; ++i) {
84                 data_[i] = other.data_[i];

```

```

85         }
86     }
87
88     return *this;
89 }
90
91 MyVector& operator=(MyVector&& other) noexcept {
92     if (this != &other) {
93         delete[] data_;
94
95         data_ = other.data_;
96         size_ = other.size_;
97         capacity_ = other.capacity_;
98
99         other.data_ = nullptr;
100        other.size_ = 0;
101        other.capacity_ = 0;
102    }
103
104    return *this;
105 }
106
107 T& operator[](std::size_t index) { return data_[index]; }
108 const T& operator[](std::size_t index) const { return data_[index]; }
109
110 std::size_t size() const { return size_; }
111 std::size_t capacity() const { return capacity_; }
112
113 void push_back(const T& value) {
114     if (size_ == capacity_) {
115         capacity_ = (capacity_ == 0 ? 1 : capacity_ * 2);
116         T* new_data = new T[capacity_];
117         for (size_t i = 0; i != size_; ++i) {
118             new_data[i] = std::move(data_[i]);
119         }
120
121         delete[] data_;
122         data_ = new_data;
123     }
124
125     data_[size_++] = value;
126 }

```

```

127
128 T* begin () { return data_; }
129 const T* begin () const { return data_; }
130 T* end () { return data_ + size_; }
131 const T* end () const { return data_ + size_; }
132
133 using reverse_iterator = std :: reverse_iterator<T*>;
134 using const_reverse_iterator = std :: reverse_iterator<const T*>;
135
136 reverse_iterator r begin () { return reverse_iterator(end()); }
137 const_reverse_iterator r begin () const { return
    const_reverse_iterator(end()); }
138 reverse_iterator r end () { return reverse_iterator(begin()); }
139 const_reverse_iterator r end () const { return const_reverse_iterator
    (begin()); }
140
141 void clear () {
142     delete [] data_;
143     data_ = new T[1]();
144     size_ = 0;
145     capacity_ = 1;
146 }
147
148 T* data () { return data_; }
149 const T* data () const { return data_; }
150 };
151
152
153 class Solution final{
154 private: //for supportive funcs
155     Solution() = delete;
156
157     template <typename T>
158     static constexpr std :: size _t get_digits_count(T num) noexcept {
159         return size of(T) * 8;
160     }
161
162 private: //for classes
163     class Digitizer final{
164     private:
165         std :: size _t idx_;
166         std :: size _t step_;
167         std :: size _t width_;

```

```

168
169     public:
170         Digitizer(std::size_t idx, std::size_t step, std::size_t width)
171             : idx_{idx}
172             , step_{step}
173             , width_{width}
174         {}
175
176         std::size_t radix() const { return 1 « step_; }
177
178         template <typename T>
179         std::size_t get_pos(T value) const {
180             return (value » idx_) & ((T{1} « step_) - T{1});
181         }
182     };
183
184     private: //for private funcs
185         template <typename InIter, typename OutIter>
186         static void counting_sort(InIter start_it, InIter end_it,
187                                   OutIter out_it, Digitizer digit)
188         {
189             std::size_t count[digit.radix()]{};
190
191             for (auto it = start_it; it != end_it; ++it) {
192                 ++count[digit.get_pos(it->date)];
193             }
194
195             for (std::size_t i = 1; i != digit.radix(); ++i) {
196                 count[i] += count[i - 1];
197             }
198
199             auto rbegin = std::reverse_iterator(end_it);
200             auto rend = std::reverse_iterator(start_it);
201             for (auto it = rbegin; it != rend; ++it) {
202                 std::size_t pos = count[digit.get_pos(it->date)] - 1;
203                 - count[digit.get_pos(it->date)];
204                 out_it += pos;
205                 *out_it = *it;
206                 out_it -= pos;
207             }
208         }
209
210     public:

```

```

211     template <typename Iterator>
212     static void radix_sort(Iterator start, Iterator end) {
213         if (start == end) { return; }
214
215         std::size_t size = static_cast<std::size_t>(std::distance(start
216             , end));
217         std::size_t idx = 0;
218         std::size_t step = 8;
219         std::size_t width = get_digits_count(start->date);
220
221         using T = typename std::iterator_traits<Iterator>::value_type;
222
223         MyVector<T> buffer( size );
224
225         bool sorted_in_buffer = false;
226         auto buffer_start = buffer.begin();
227         auto buffer_end = buffer.end();
228
229         while (idx < width) {
230             Digitizer digit{idx, step, width};
231
232             if (!sorted_in_buffer) {
233                 counting_sort(start, end, buffer_start, digit);
234                 sorted_in_buffer = true;
235             } else {
236                 counting_sort(buffer_start, buffer_end, start, digit);
237                 sorted_in_buffer = false;
238             }
239
240             idx += step;
241         }
242
243         if (sorted_in_buffer) {
244             auto buffer_it = buffer_start;
245             for (auto it = start; it != end; ++it) {
246                 *it = *buffer_it++;
247             }
248         }
249     };
250
251

```



```

252 std :: size_t find(const MyVector<char>& line, char c, std :: size_t start
    = 0) {
253     for (std :: size_t i = start; i != line.size(); ++i) {
254         if (line[i] == c) {
255             return i;
256         }
257     }
258
259     return -1;
260 }
261
262 MyVector<char> substr(const MyVector<char>& line, std :: size_t start,
    std :: size_t end) {
263     MyVector<char> result;
264     for (std :: size_t i = start; i != start + end; ++i) {
265         result.push_back(line[i]);
266     }
267
268     return result;
269 }
270
271 int stoi(const MyVector<char>& line) {
272     int result = 0;
273     for (std :: size_t i = 0; i != line.size(); ++i) {
274         result = result * 10 + (line[i] - '0');
275     }
276
277     return result;
278 }
279
280 Item parser(const MyVector<char>& line, std :: size_t ind) {
281     std :: size_t pos1 = find(line, '.');
282     std :: size_t pos2 = find(line, '.', pos1 + 1);
283     std :: size_t pos3 = find(line, '\\t');
284
285     uint8_t day = static_cast<uint8_t>(stoi(substr(line, 0, pos1)));
286     uint8_t month = static_cast<uint8_t>(stoi(substr(line, pos1 + 1,
        pos2 - pos1 - 1)));
287     uint16_t year = static_cast<uint16_t>(stoi(substr(line, pos2 + 1,
        pos3 - pos2 - 1)));
288     uint32_t date = static_cast<uint32_t>(year) * 10000 +
289                     static_cast<uint32_t>(month) * 100 +
290                     static_cast<uint32_t>(day);

```

```

291     uint32_t pos = static_cast<uint32_t>(ind);
292     uint32_t len = static_cast<uint32_t>(line.size());
293
294     return Item{date, pos, len};
295 }
296
297
298 int main() {
299     MyVector<Item> vec;
300     MyVector<char> all_data;
301     uint32_t pos = 0;
302
303     char c;
304     MyVector<char> line;
305     bool line_feed_flag = false;
306     while (std::cin.get(c)) {
307         if (c == '\\n') {
308             if (line.size() > 0) {
309                 vec.push_back(parser(line, pos));
310                 line.clear();
311                 pos = all_data.size();
312             }
313
314             } else {
315                 all_data.push_back(c);
316                 line.push_back(c);
317             }
318         }
319
320         if (line.size() > 0) {
321             vec.push_back(parser(line, pos));
322             line.clear();
323         }
324
325         Solution::radix_sort(vec.begin(), vec.end());
326
327         for (auto& elem : vec) {
328             std::cout.write(all_data.data() + elem.pos, elem.len);
329             std::cout << "\\n";
330         }
331
332         return 0;
333     }

```

6 Тесты

Входные данные:

15.03.22 12345
01.01.23 7890
07.11.21 42
31.12.22 999999
22.02.22 777
09.05.23 2023
13.06.21 100500
25.10.22 8
03.07.23 55555
18.08.21 13

Выходные данные:

13.06.21	100500
18.08.21	13
07.11.21	42
22.02.22	777
15.03.22	12345
25.10.22	8
31.12.22	999999
01.01.23	7890
09.05.23	2023
03.07.23	55555

7 Вывод

Асимптотика по времени в среднем составляет $O(n + k) * m$, где n - количество элементов, k - диапазон значений ключа, m - число разрядов. Таким образом, асимптотика составляет $O(n + k)$, так как m - константа (количество разрядов задается автоматически программой, 8). Асимптотика по памяти: $O(n)$, где n — число элементов.